

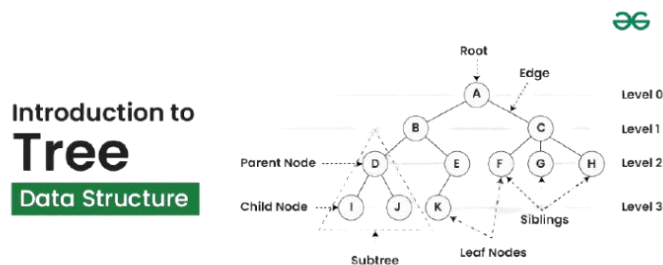
Introduction to Tree Data Structure

Tree data structure is a hierarchical structure that is used to represent and organize data in a way that is easy to navigate and search. It is a collection of nodes that are connected by edges and has a hierarchical relationship between the nodes.

The topmost node of the tree is called the **root**, and the nodes below it are called the child nodes. Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.

Why Tree is considered a non-linear data structure?

The data in a tree are not stored in a sequential manner i.e., they are not stored linearly. Instead, they are arranged on multiple levels or we can say it is a hierarchical structure. For this reason, the tree is considered to be a non-linear data structure.



Basic Terminologies In Tree Data Structure:

Root Node: The topmost node of a tree or the node which does not have any parent node is called the root node. {A} is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree. All other nodes branch out from this node.

Child: A child node is a node directly connected to another node (its parent).

A
/
\
B C

Here, B and C are children of A.

Parent: A parent is a node that has one or more children.

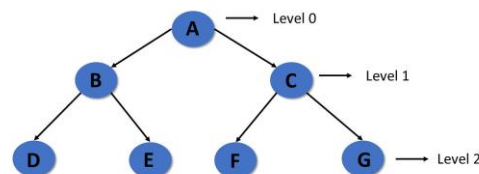
Leaf Node or External Node: The nodes which do not have any child nodes are called leaf nodes. {I, J, K, F, G, H} are the leaf nodes of the tree from the above diagram.

Sibling: Children of the same parent node are called siblings. {D,E} are called siblings.

Ancestor and Descendant:

- An ancestor of a node is any node along the path from the root to that node.
- A descendant of a node is any node that can be reached by following edges downwards.

Level of a node: The count of edges on the path from the root node to that node. The root node has level 0.

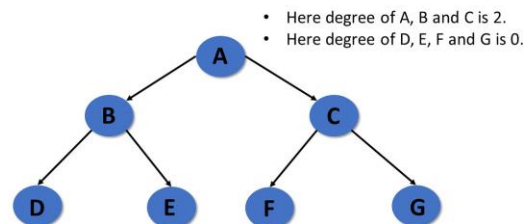


Internal node: A node with at least one child is called Internal Node.

Edge: An edge is the connection between two nodes. It represents the relationship between parent and child.

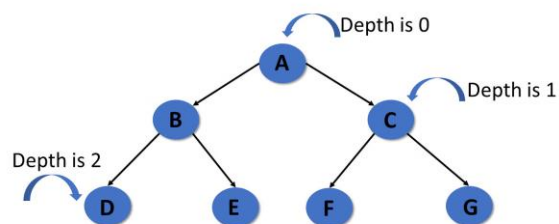
Degree:

- The degree of a node refers to the number of children it has.
- Example: If a node has two children, its degree is 2.



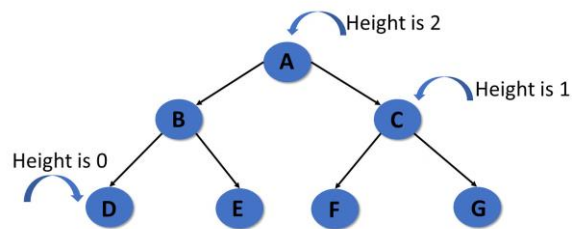
Depth:

- The depth of a node is the number of edges from the root to the node.
- Example: A node that is two levels below the root has a depth of 2.

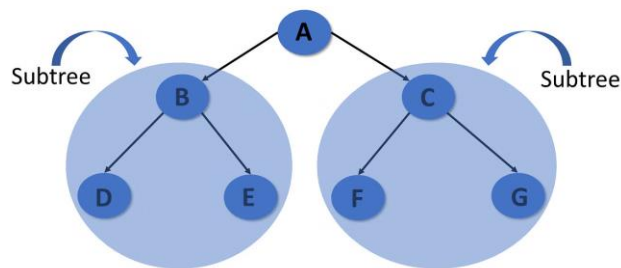


Height:

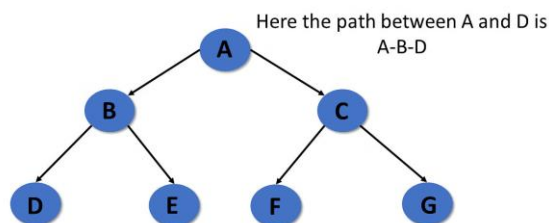
- The height of a node is the number of edges from that node to the deepest leaf node.
- The height of the tree is the height of the root node.
- Example: In a tree with three levels, the height is 2.

**Subtree:**

- A subtree is any tree that is part of a larger tree. It consists of a node and all its descendants.

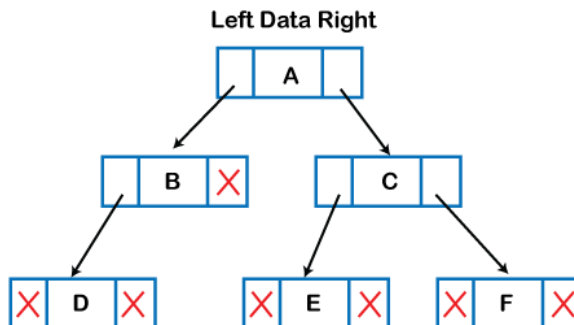
**Path:**

- A path is a sequence of nodes and edges connecting a node to its descendant.
- Example: In a decision tree, the path from the root to a leaf represents a sequence of decisions.



Representation of a Node in Tree Data Structure:

A tree can be represented using a collection of nodes. Each of the nodes can be represented with the help of structs. The tree data structure can be created by creating the nodes dynamically with the help of the pointers. The tree in the memory can be represented as shown below:



```

struct Node {
    int data;      // Data part
    struct Node *left; // Pointer to left child
    struct Node *right; // Pointer to right child
};
  
```

Importance for Tree Data Structure:

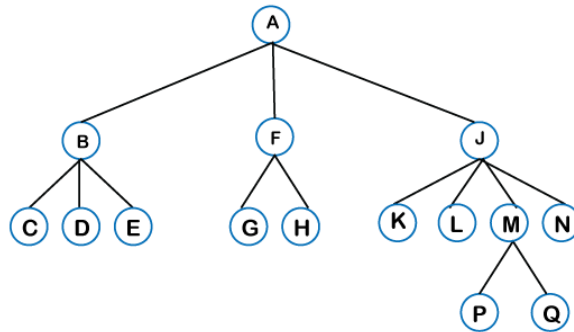
One reason to use trees might be because you want to store information that naturally forms a hierarchy. For example, the file system on a computer

Types of Tree data structure

The following are the types of a tree data structure:

- **General tree:** The general tree is one of the types of tree data structure. In the general tree, a node can have either 0 or maximum n number of nodes. There is no restriction imposed on the degree of the node (the number of nodes that a node can contain). The topmost node in a general tree is

known as a root node. The children of the parent node are known as **subtrees**.



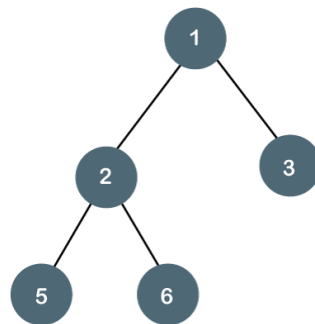
There can be n number of subtrees in a general tree. In the general tree, the subtrees are unordered as the nodes in the subtree cannot be ordered.

- **Binary Tree:** A **binary tree** is a type of tree where each node can have up to two child nodes. This means a node in a binary tree can either have no children, one child, or two children. The term "binary" refers to this specific limit of having at most two children per node.
- **Binary Search Tree (BST):** A binary tree with additional constraints:
 - The left child contains values less than the parent.
 - The right child contains values greater than the parent.
- **AVL Tree:** A self-balancing binary search tree where the height difference between the left and right subtrees of any node is at most 1.

Binary Tree

A **binary tree** is a type of tree where each node can have up to two child nodes. This means a node in a binary tree can either have no children, one child, or two children. The term "binary" refers to this specific limit of having at most two children per node. Every node consists of:

1. **Data:** The value stored in the node.
2. **Pointer to the left child:** Points to the left child node (if it exists).
3. **Pointer to the right child:** Points to the right child node (if it exists).



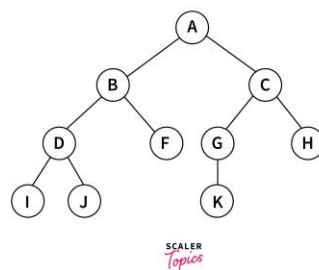
The above tree is a binary tree because each node contains the utmost two children.

Types of Binary Trees

1. **Full Binary Tree:** Every node either has **0 or 2 children**. No node has only one child.
2. **Complete Binary Tree:** All levels except possibly the last are completely filled, and all nodes in the last level are as far left as possible.
3. **Perfect Binary Tree:** A binary tree in which all internal nodes have exactly two children, and all leaf nodes are at the same level.
4. **Balanced Binary Tree:** The height difference between the left and right subtrees of every node is at most one.
5. **Degenerate (or Skewed) Tree:** Every parent node has only one child. This resembles a linked list.

Binary Tree Representation

A binary tree can be represented using a linked list and also using an array.

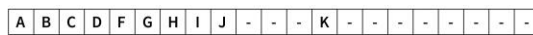


Array Representation :

The binary tree can be represented using an array of size $2n+1$ if the depth of the binary tree is n . If the parent element is at the index p , Then the left child will be stored in the index $(2p)+1$, and the right child will be stored in the index $(2p)+2$.

The array representation of the above binary tree is :

As in the above binary tree, A was the root node, so that it will be stored in the 0th index. The left child of A will be stored in the $2(0)+1$ equal to the 1st location. So, B is stored in index 1. And similarly, the right child of A will be stored in the $2(0)+2$ index. For every node, the left and right child will be stored accordingly.

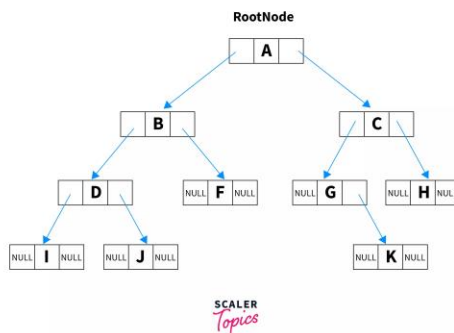


SCALER Topics

Linked List Representation :

For the linked list representation, we will use the doubly linked list, which has two pointers so that we can point to the left and right children of a binary tree node. NULL is given to the pointer as the address when no child is connected.

The Linked list representation of the above binary tree is :



SCALER Topics

Need for Binary Trees

- Searching becomes easy and faster with the help of a binary search tree, even for huge data sets.
- Trees do not have a limit on the nodes. So, according to the user's need, a tree of any size can be created and further increased, unlike an array.
- It can store hierarchical data in the database.

- A binary tree can also be used for classification and decision-making.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int item;
    struct node* left;
    struct node* right;
};

// Inorder traversal
void inorderTraversal(struct node* root) {
    if (root == NULL) return;
    inorderTraversal(root->left);
    printf("%d ", root->item);
    inorderTraversal(root->right); }

// Preorder traversal
void preorderTraversal(struct node* root) {
    if (root == NULL) return;
    printf("%d ", root->item);
    preorderTraversal(root->left);
    preorderTraversal(root->right);
}

// Postorder traversal
void postorderTraversal(struct node* root) {
    if (root == NULL) return;
    postorderTraversal(root->left);
    postorderTraversal(root->right);
    printf("%d ", root->item);
}

// Create a new Node
```

```
struct node* create(int value) {  
    struct node* newNode = malloc(sizeof(struct node));  
    newNode->item = value;  
    newNode->left = NULL;  
    newNode->right = NULL;  
    return newNode;  
}
```

// Insert on the left of the node

```
struct node* insertLeft(struct node* root, int value) {  
    root->left = create(value);  
    return root->left; }  
  
// Insert on the right of the node
```

```
struct node* insertRight(struct node* root, int value) {  
    root->right = create(value);  
    return root->right;  
}
```

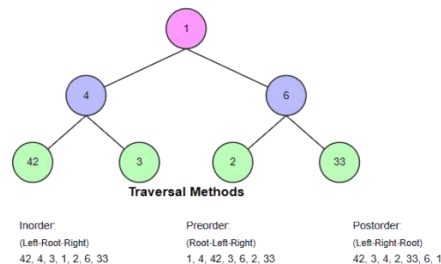
```
int main() {  
    struct node* root = create(1);  
    insertLeft(root, 4);  
    insertRight(root, 6);  
    insertLeft(root->left, 42);  
    insertRight(root->left, 3);  
    insertLeft(root->right, 2);  
    insertRight(root->right, 33);  
    printf("Traversal of the inserted binary tree \n");  
    printf("Inorder traversal \n");  
    inorderTraversal(root);  
    printf("\nPreorder traversal \n");  
    preorderTraversal(root);  
}
```

```

printf("\nPostorder traversal \n");
postorderTraversal(root);

return 0;
}

```



Applications of trees

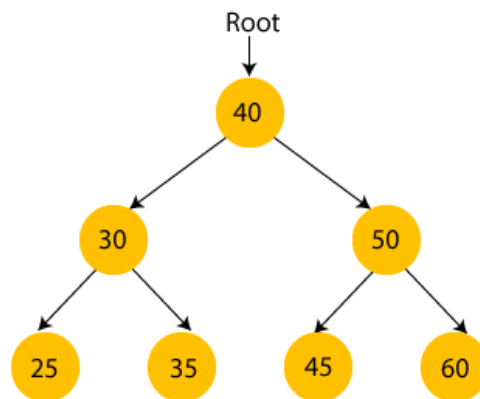
The following are the applications of trees:

- **Storing naturally hierarchical data:** Trees are used to store the data in the hierarchical structure. For example, the file system. The file system stored on the disc drive, the file and folder are in the form of the naturally hierarchical data and stored in the form of trees.
- **Organize data:** It is used to organize data for efficient insertion, deletion and searching. For example, a binary tree has a $\log N$ time for searching an element.
- **Trie:** It is a special kind of tree that is used to store the dictionary. It is a fast and efficient way for dynamic spell checking.
- **Heap:** It is also a tree data structure implemented using arrays. It is used to implement priority queues.
- **B-Tree and B+Tree:** B-Tree and B+Tree are the tree data structures used to implement indexing in databases.
- **Routing table:** The tree data structure is also used to store the data in routing tables in the routers.

Binary Search Tree:

A **Binary Search Tree (BST)** is a node-based hierarchical data structure in which each node has at most two children, referred to as the left child and the right child. It is organized according to the following properties:

1. The value of the left child of a node is always **less than** the value of the node.
2. The value of the right child of a node is always **greater than** the value of the node.
3. The left and right subtrees of each node must also be binary search trees.



Advantages of Binary search tree

Searching an element in the Binary search tree is easy as we always have a hint that which subtree has the desired element.

- As compared to array and linked lists, insertion and deletion operations are faster in BST.

Search Operation

Algorithm for searching an element in a binary tree is as follows:

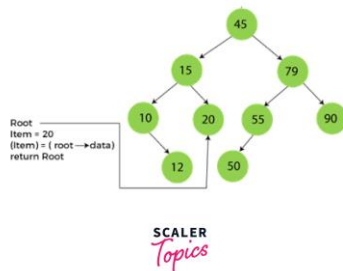
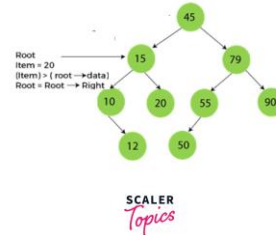
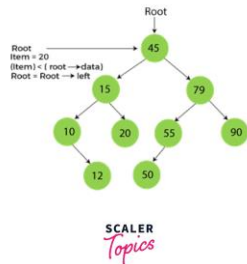
1. Compare the element to be searched with the root node of the tree.
2. If the value of the element to be searched is equal to the value of the root node, return the root node.
3. If the value does not match, check whether the value is less than the root element or not if it is then traverse the left subtree.
4. If larger than the root element, traverse the right subtree.
5. If the element is not found in the whole tree, return NULL.

Algorithm:

```

If root == NULL
    return NULL;
If number == root->data
    return root->data;
If number < root->data
    return search(root->left)
If number > root->data
    return search(root->right)

```



Insert Operation

Inserting a value in the correct position is similar to searching because we try to maintain the rule that the left subtree is lesser than root and the right subtree is larger than root.

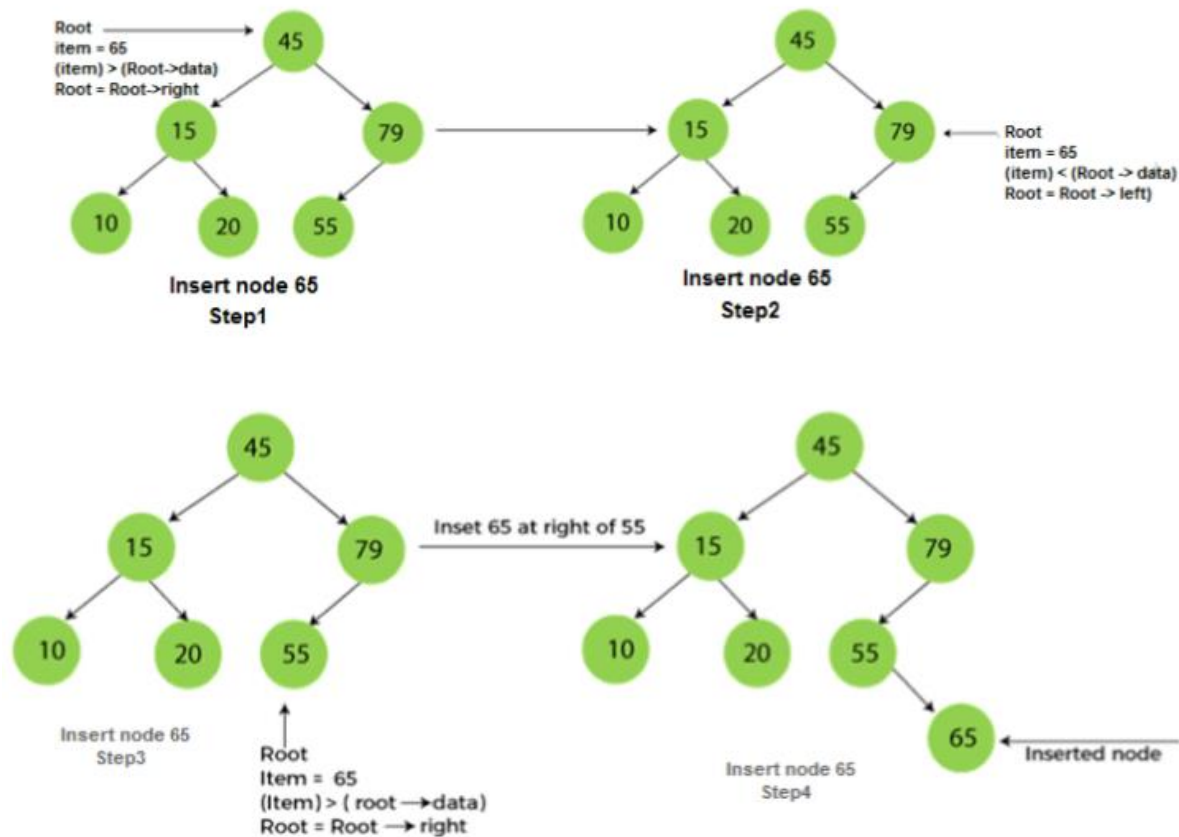
We keep going to either right subtree or left subtree depending on the value and when we reach a point left or right subtree is null, we put the new node there.

Algorithm:

```

If node == NULL
    return createNode(data)
if (data < node->data)
    node->left = insert(node->left, data);
else if (data > node->data)
    node->right = insert(node->right, data);
return node;

```



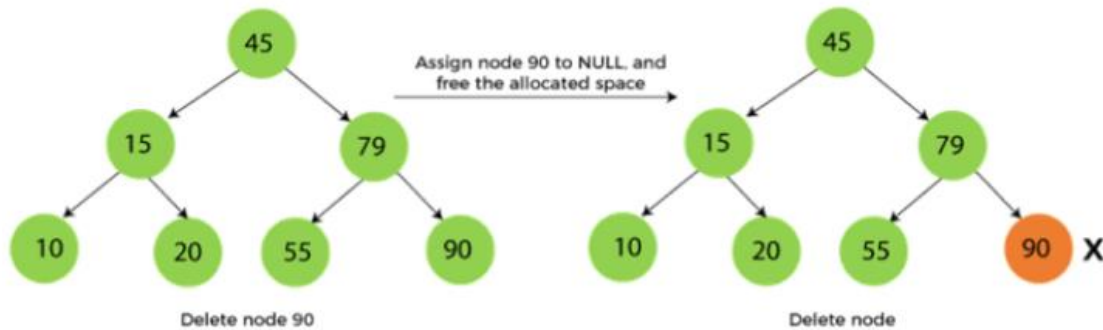
Deletion Operation

In the deletion operation, we have to delete a node from the binary search tree in a way that does not violate its properties. Deletion can occur in three possible cases:

1. Node to be deleted is the leaf node

This is the simplest case of deleting a node in a binary search tree. Here, we will replace the leaf node with NULL and free the allocated space.

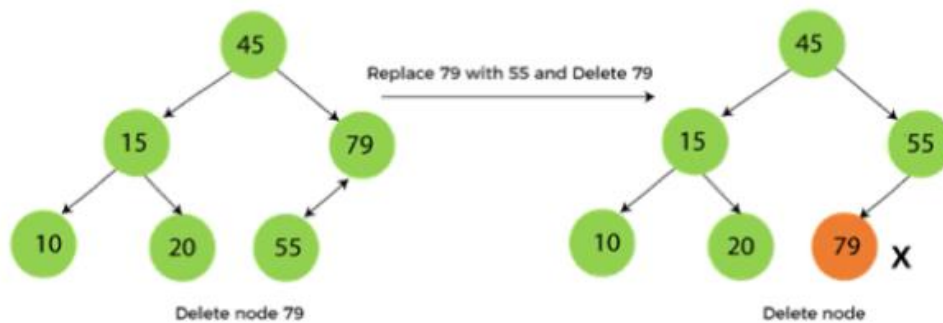
We can understand the deletion operation in a binary search tree with the help of an example where we are trying to delete the node having a value of 90



2. Node to be deleted has a single child node

In the second case, the node to be deleted has a single child node. In such a case follow the steps below:

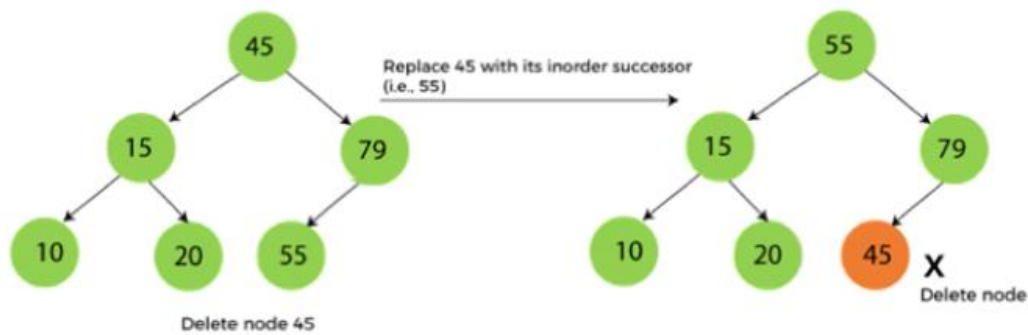
1. Replace that node with its child node.
2. Remove the child node from its original position.



3. The node to be deleted has two children

In the third case, the node to be deleted has two children. In such a case follow the steps below:

1. Get the inorder successor of that node.
2. Replace the node with the inorder successor.
3. Remove the inorder successor from its original position.



CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int data; //node will store some data
```

```
    struct node *right_child; // right child
```

```
    struct node *left_child; // left child
```

```
};
```

//function to create a node

```
struct node* new_node(int x) {
```

```
    struct node *temp;
```

```
    temp = malloc(sizeof(struct node));
```

```
    temp -> data = x;
```

```
    temp -> left_child = NULL;
```

```
    temp -> right_child = NULL;
```

```
    return temp;
```

```
}
```


// searching operation

```
struct node* search(struct node * root, int x) {  
    if (root == NULL || root -> data == x) //if root->data is x then the element is found  
        return root;  
    else if (x > root -> data) // x is greater, so we will search the right subtree  
        return search(root -> right_child, x);  
    else //x is smaller than the data, so we will search the left subtree  
        return search(root -> left_child, x);  
}
```

// insertion

```
struct node* insert(struct node * root, int x) {  
    //searching for the place to insert  
    if (root == NULL)  
        return new_node(x);  
    else if (x > root -> data) // x is greater. Should be inserted to the right  
        root -> right_child = insert(root -> right_child, x);  
    else // x is smaller and should be inserted to left  
        root -> left_child = insert(root -> left_child, x);  
    return root;  
}
```

//function to find the minimum value in a node

```
struct node* find_minimum(struct node * root) {  
    if (root == NULL)  
        return NULL;  
    else if (root -> left_child != NULL) // node with minimum value will have no left child  
        return find_minimum(root -> left_child); // left most element will be minimum
```

```
return root; }
```

// deletion

```
struct node* delete(struct node * root, int x) {  
    //searching for the item to be deleted  
    if (root == NULL)  
        return NULL;  
    if (x > root->data)  
        root->right_child = delete(root->right_child, x);  
    else if (x < root->data)  
        root->left_child = delete(root->left_child, x);  
    else {  
        //No Child node  
        if (root->left_child == NULL && root->right_child == NULL) {  
            free(root);  
            return NULL;  
        }  
  
        //One Child node  
        else if (root->left_child == NULL || root->right_child == NULL) {  
            struct node *temp;  
            if (root->left_child == NULL)  
                temp = root->right_child;  
            else  
                temp = root->left_child;  
            free(root);  
            return temp;  
        }  
    }  
}
```

```
//Two Children
else {
    struct node *temp = find_minimum(root -> right_child);
    root -> data = temp -> data;
    root -> right_child = delete(root -> right_child, temp -> data);
}
}
return root;
}
```

// Inorder Traversal

```
void inorder(struct node *root) {
    if (root != NULL) // checking if the root is not null
    {
        inorder(root -> left_child); // traversing left child
        printf(" %d ", root -> data); // printing data at root
        inorder(root -> right_child); // traversing right child
    }
}
```

```
int main() {
    struct node *root;
    root = new_node(20);
    insert(root, 5);
    insert(root, 1);
    insert(root, 15);
    insert(root, 9);
}
```

```
insert(root, 7);
insert(root, 12);
insert(root, 30);
insert(root, 25);
insert(root, 40);
insert(root, 45);
insert(root, 42);

inorder(root);
printf("\n");

root = delete(root, 1);

root = delete(root, 40);

root = delete(root, 45);

root = delete(root, 9);

inorder(root);
printf("\n");

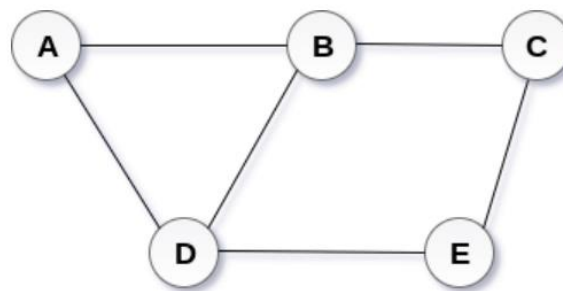
return 0;
}
```

What is Graph Data Structure?

Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally:

A graph G can be defined as an ordered set $G(V, E)$ where $V(G)$ represents the set of vertices and $E(G)$ represents the set of edges which are used to connect these vertices.

A Graph $G(V, E)$ with 5 vertices (A, B, C, D, E) and six edges ((A,B), (B,C), (C,E), (E,D), (D,B), (D,A)) is shown in the following figure.



Terminology Related to Graphs

1. **Vertex (Node):** A fundamental unit in a graph that represents an object or entity. Denoted as V or v in mathematical terms.
2. **Edge:** A connection between two vertices that shows a relationship between them. Denoted as E or e .
3. **Adjacency:** Two vertices are said to be adjacent if there is an edge connecting them.
4. **Degree of a Vertex:** The number of edges connected to a vertex.
 - **In-degree:** The number of incoming edges (for directed graphs).
 - **Out-degree:** The number of outgoing edges (for directed graphs).
5. **Path:** A path in a graph is a finite or infinite set of edges which joins a set of vertices. It can connect to 2 or more nodes. Also, if the path connects all the nodes of a

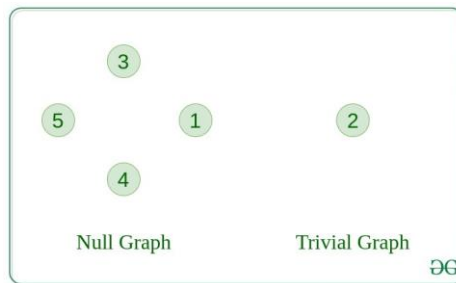
graph in Data Structure, then it is a connected graph, otherwise it is called a disconnected graph.

6. **Loop:** A loop (also called a self-loop) is an edge that connects a vertex to itself. It is commonly defined as an edge with both ends as the same vertex.

Types Of Graphs in Data Structure and Algorithms

1. **Null Graph :** A graph is known as a null graph if there are no edges in the graph.

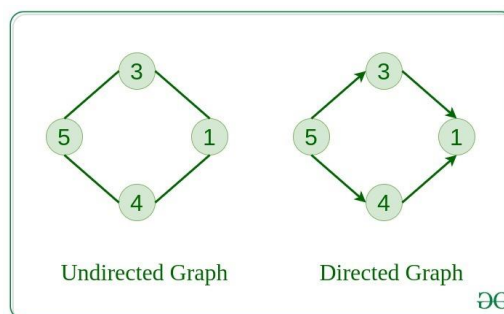
2. **Trivial Graph:** Graph having only a single vertex, it is also the smallest graph possible.



3. Undirected Graph

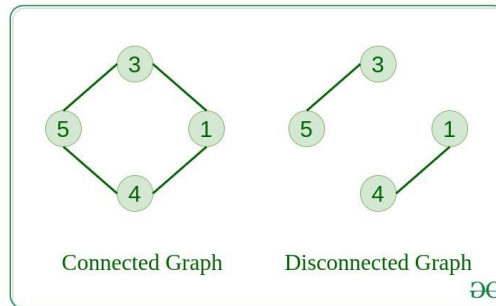
A graph in which edges do not have any direction. That is the nodes are unordered pairs in the definition of every edge.

4. **Directed Graph :** A graph in which edge has direction. That is the nodes are ordered pairs in the definition of every edge.



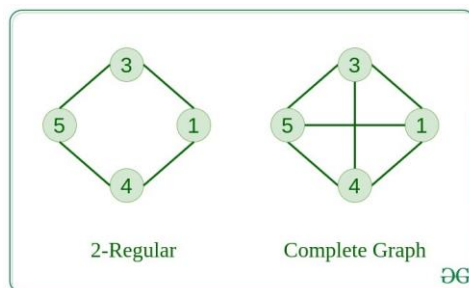
5. **Connected Graph:** The graph in which from one node we can visit any other node in the graph is known as a connected graph.

6. **Disconnected Graph:** The graph in which at least one node is not reachable from a node is known as a disconnected graph.



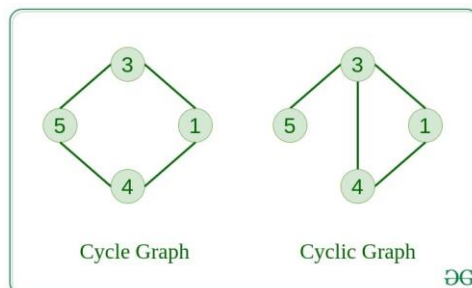
7. Regular Graph: The graph in which the degree of every vertex is equal to K is called K regular graph.

8. Complete Graph: The graph in which from each node there is an edge to each other node .



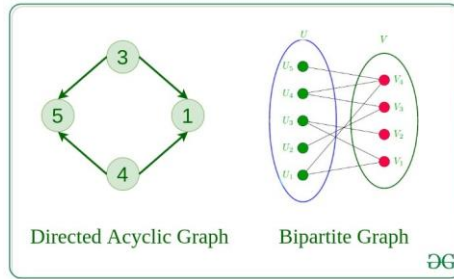
9. Cycle Graph: The graph in which the graph is a cycle in itself, the minimum value of degree of each vertex is 2.

10. Cyclic Graph: A graph containing at least one cycle is known as a Cyclic graph.



11. Directed Acyclic Graph: A Directed Graph that does not contain any cycle.

12. Bipartite Graph: A graph in which vertex can be divided into two sets such that vertex in each set does not contain any edge between them.



13. Weighted Graph: A graph in which the edges are already specified with suitable weight is known as a weighted graph.

Weighted graphs can be further classified as directed weighted graphs and undirected weighted graphs.

Graph representation

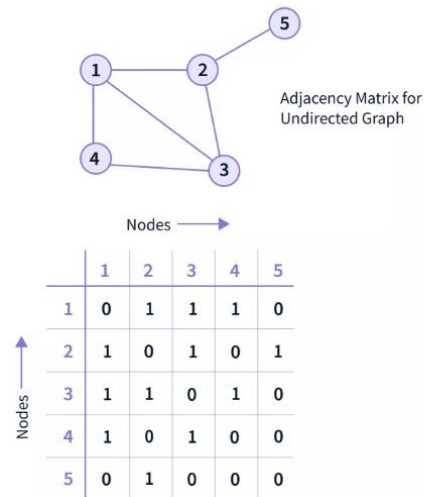
By Graph representation, we simply mean the technique to be used to store some graph into the computer's memory.

- **Sequential representation** (or, Adjacency matrix representation)
- **Linked list representation** (or, Adjacency list representation)

Adjacency Matrix

An Adjacency Matrix is a 2D array of size $V \times V$ where V is the number of nodes in a graph. It is used to represent a "finite graph", with 0's and 1's. Since, its size is $V \times V$, it is a square matrix.

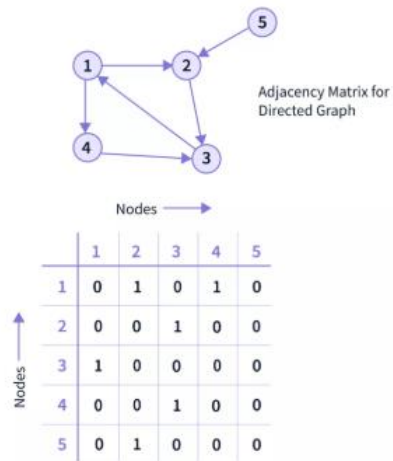
The elements of the matrix indicate whether pairs of vertices are adjacent or not in the graph i.e. is there any edge connecting a pair of nodes in the graph.



The above example shows the adjacency matrix for the undirected graph. Let us note some important points:

1. The adjacent matrix's row or column, consists of the nodes or vertices(that is numbered in red, in the above graph).
2. Each cell in the above matrix is represented as A_{ij} , where i and j are nodes. The value of A_{ij} is either 1 or 0 depending on whether there is an edge from vertex i to vertex j . If there is a node then, $A_{ij} = 1$, otherwise $A_{ij} = 0$
3. Adjacency matrix of an undirected graph is symmetric. Hence, the above graph is symmetric.

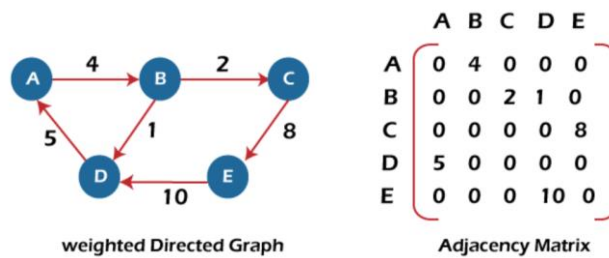
The adjacency Matrix for a directed graph also follows the same conventions, except for, there is a '1' in the matrix if there is an edge pointing from one node to another, say from node A to node B. But vice versa may not be applicable. That is, in a directed graph, if $A[i][j] = 1$ then $A[j][i]$ may or may not be 1. Adjacency matrix of a directed graph is not symmetric(generally).



Adjacency Matrix for Weighted Graphs

If the graph is weighted, then we usually call the matrix as the cost matrix. To store weighted graph using adjacency matrix form, we follow the following steps:

- Here each cell at position $A[i, j]$ holds the weight from edge i to j .
- If the edge is not present, then it stores infinity or any largest value(which cannot be the weight of any node in the graph).
- For same node, the value in the matrix is 0.
- Apart from this, the rest of the steps are similar for the adjacency matrix of the graph.



Code:

```
#include<stdio.h>

#define V 4

void addEdge(int mat[V][V], int i, int j, int isDirected) {
    mat[i][j] = 1; // Add edge from i to j
    if (!isDirected) {
        mat[j][i] = 1; // Add reverse edge for undirected graphs
    }
}

void displayMatrix(int mat[V][V]) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++)
            printf("%d ", mat[i][j]);
        printf("\n");
    }
}

int main() {
    int mat[V][V] = {0}; // Initialize all edges to 0

    // Set graph type (0 for undirected, 1 for directed)
    int isDirected = 1; // Change to 0 for undirected graphs

    // Add edges
    addEdge(mat, 0, 1, isDirected);
    addEdge(mat, 0, 2, isDirected);
    addEdge(mat, 1, 2, isDirected);
    addEdge(mat, 2, 3, isDirected);

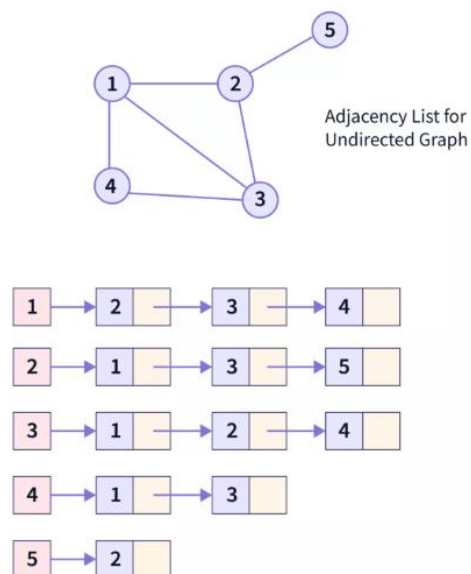
    printf("Adjacency Matrix Representation\n");
```

```
displayMatrix(mat);  
return 0; }
```

Adjacency List

- An adjacency list represents a graph as an array of linked lists.
- The index of the array represents a node.
- Each element in the linked list represents the nodes that are connected to that node by an edge.

Let us take an example for easy visualization --



CODE:

Breadth First Search or BFS for a Graph

Breadth First Search (BFS) is a fundamental **graph traversal algorithm**. It begins with a node, then first traverses all its adjacent. Once all adjacent are visited, then their adjacent are traversed. This is different from DFS in a way that closest vertices are visited before others. We mainly traverse vertices level by level.

BFS can be used to detect cycle in a directed and undirected graph, find shortest path in an unweighted graph and many more problems.

BFS from a Given Source:

The algorithm starts from a given source and explores all reachable vertices from the given source.

we begin with the given source and traverse vertices level by level using a queue data structure. The only catch here is that, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a **boolean visited** array.

Initialization: Enqueue the given source vertex into a queue and mark it as visited.

1. **Exploration:** While the queue is not empty:
 - a. Dequeue a node from the queue and visit it (e.g., print its value).
 - b. For each unvisited neighbor of the dequeued node:
 - i. Enqueue the neighbor into the queue.
 - ii. Mark the neighbor as visited.
2. **Termination:** Repeat step 2 until the queue is empty.

This algorithm ensures that all nodes in the graph are visited in a breadth-first manner, starting from the starting node.

The BFS algorithm's pseudo-code can be implemented in the following way:

```
BFS(Graph, start_vertex):  
    create a queue Q and enqueue start_vertex into it  
    create a set (or list) V to track visited vertices  
    while Q is not empty:  
        current_vertex = Q.dequeue()  
        if current_vertex is not in V:  
            add current_vertex to V  
            for every neighbor of current_vertex:  
                enqueue neighbor into Q
```

Time Complexity: $O(V+E)$, where V is the number of nodes and E is the number of edges.

Auxiliary Space: $O(V)$

Depth-First Search (DFS)

Depth-First Search (DFS) is an algorithm used to traverse or search a graph in a depthward motion. It uses a stack data structure to remember to get the next vertex to start a search when a dead-end occurs in any iteration.

Here's a high-level view of how it works:

- Start from the root or any arbitrary node (this typically becomes the 'source' node).
- Explore the adjacent nodes of the current node.
- Go deep into the graph by visiting the next adjacent node.
- When you visit a node that has no adjacent nodes that you haven't visited yet, you go back to the previous node and visit another node from its adjacent nodes.
- Repeat the steps until all nodes are visited.

The DFS algorithm's pseudo-code can be implemented in the following way:

```
DFS(Graph, start_vertex):  
    create a stack S and push start_vertex into it  
    create a set (or list) V to track visited vertices  
    while S is not empty:  
        current_vertex = S.pop()  
        if current_vertex is not in V:  
            add current_vertex to V  
            for every neighbor of current_vertex:  
                push neighbor into S
```

APPLICATIONS OF GRAPHS:

Network Routing: Graphs are used to model computer networks, where vertices represent devices (routers, switches, computers) and edges represent connections (cables or wireless links).

Social Networks: Platforms like Facebook, Instagram, or LinkedIn use graphs to model relationships between users (vertices) and friendships or connections (edges).

Web Crawling and Page Ranking: Search engines represent the web as a directed graph, where vertices are web pages and edges are hyperlinks.

Road Networks: Graphs can be used to model transportation systems, such as roads and flights, and to find the shortest or quickest routes between locations. Vertices represent cities or intersections, and edges represent roads.

Genomics and Protein Interaction Networks: Graphs are used to model gene interactions, protein-protein networks, or metabolic pathways.

Graph Neural Networks: Machine learning models use graphs to capture relationships in structured data like social networks or chemical compounds.

Game Trees: Graphs are used to represent possible moves in games like chess or tic-tac-toe.

Scheduling: Directed Acyclic Graphs (DAGs) are used for task scheduling in project management (e.g., PERT and CPM).